Bab 2 Fondasi Aplikasi Web: Modular Monolith, Routing, Controller, dan Lapisan View (Blade)

Tujuan Pembelajaran

Setelah mempelajari bab ini, Anda mampu:

1. Menjelaskan mekanisme *routing*, *controller*, *Blade Template*, serta siklus *request–response–session* pada Laravel.
2. Mengimplementasikan *routing* berparameter, *controller*, layout berlapis, dan formulir menggunakan *Blade Template*.
3. Menerapkan pengelolaan *request*, *middleware*, *response*, dan *session* pada aplikasi web.
4. Menganalisis pembagian tanggung jawab antar-lapisan berdasarkan prinsip *separation of concerns*.
5. Merancang struktur *modular monolith* berbasis *bounded context* dan tata letak antarmuka multi-peran.

2.1 Mengapa Fondasi Arsitektur Diperlukan

Aplikasi web tidak hanya terdiri atas halaman dan formulir. Di belakang setiap halaman terdapat alur yang menghubungkan URL, *request* pengguna, pemeriksaan akses, logika aplikasi, pengambilan data, dan respons yang dikirim kembali ke browser.

Tanpa pembagian tanggung jawab, *controller* dapat menjadi terlalu besar, *view* memuat logika bisnis, dan aturan keamanan tersebar pada banyak tempat. Kondisi tersebut menyulitkan pengujian, perbaikan, dan pengembangan fitur baru.

Bayangkan aplikasi Kantin Multi-Tenant. Pelanggan perlu melihat katalog dan membuat pesanan. Operator tenant perlu mengelola menu serta memantau antrean dapur. Admin perlu mengelola tenant, meja, komisi, dan laporan. Jika seluruh fitur dicampurkan tanpa batas yang jelas, perubahan pada satu bagian dapat memengaruhi bagian lain dan meningkatkan risiko kesalahan akses.

Fondasi arsitektur diperlukan untuk menjawab tiga pertanyaan berikut.

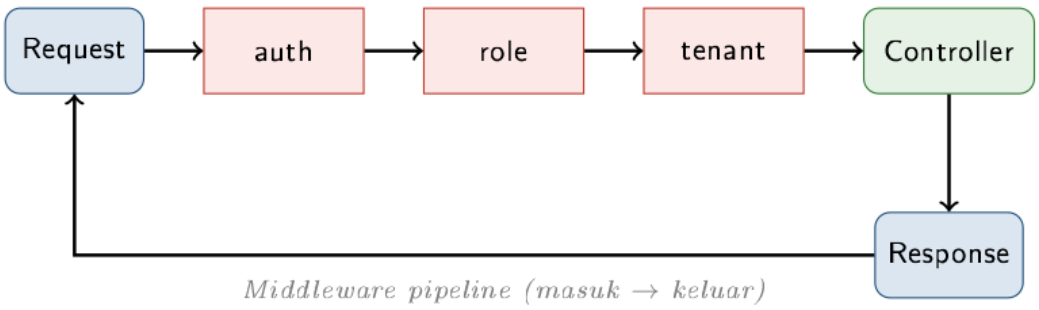
1. Bagian mana yang bertanggung jawab atas suatu aturan bisnis?
2. Bagian mana yang boleh mengakses fitur atau data tertentu?
3. Bagaimana *request* pengguna bergerak secara konsisten dari URL hingga menjadi respons?

Jawaban atas pertanyaan tersebut dibangun melalui tiga prinsip.

- **Pemisahan urusan** (*separation of concerns*): setiap bagian aplikasi menangani tanggung jawab yang jelas.
- **Batas modul**: fitur yang terkait ditempatkan dalam modul dengan batas yang dapat dipahami.
- **Pertahanan berlapis**: akses tidak hanya disembunyikan di navigasi, tetapi diperiksa juga pada *route*, *middleware*, dan aturan otorisasi di server.

2.2 Gambaran Alur Aplikasi Web Laravel

Sebelum membahas struktur modul, Anda perlu memahami perjalanan sebuah *request*. Pemahaman ini membantu Anda menentukan tempat yang tepat untuk menulis kode. Secara sederhana, alur aplikasi web Laravel dapat digambarkan pada Gambar 2.1: *request* melewati *route* dan *middleware*, ditangani *controller* yang memanggil *service* atau *model*, lalu menghasilkan *view* atau *response*.



Gambar 2.1. Pipeline request: request menuju controller dan respons kembali kepada pengguna.

Respons kemudian kembali ke pengguna. Pada proses tertentu, aplikasi juga dapat membaca atau menulis data pada *session*.

2.2.1 Request sebagai titik masuk

Request adalah permintaan HTTP dari klien ke server. Request dapat berisi metode HTTP, URL, *header*, data formulir, parameter kueri, atau data JSON. Contohnya, ketika pengguna membuka URL `/menu?q=kopi`, browser mengirim *request* GET ke *endpoint* `/menu` dengan parameter kueri `q` bernilai `kopi`. Di Laravel, *controller* dapat membaca nilai tersebut melalui objek `Request`, misalnya `$q = $request->query('q');`.

```
▌ Membaca parameter kueri dari objek Request
// URL yang dibuka pengguna: /menu?q=kopi
$q = $request->query('q');          // 'kopi'
$q = $request->query('q', 'semua'); // nilai bawaan jika parameter tidak ada
$semua = $request->all();           // seluruh masukan, belum tervalidasi
```

> **Peringatan.** Jangan menganggap data dari URL, formulir, *cookie*, *header*, atau JSON sebagai data yang aman. Semua masukan dari pengguna perlu divalidasi sesuai aturan bisnis sebelum diteruskan ke lapisan lain.

2.2.2 Route menentukan tujuan request

Route adalah pemetaan antara metode HTTP dan URL ke tindakan tertentu. Tindakan itu dapat berupa *closure* sederhana atau metode pada *controller*. Nama *route* penting karena kode aplikasi dapat membuat tautan berdasarkan nama tersebut, bukan berdasarkan URL fisik. Contohnya, `route('tenant.menus.show', [$tenant, $menu])` menghasilkan URL untuk menu tertentu. Jika struktur URL berubah, Anda cukup memperbarui definisi *route* tanpa harus mencari seluruh tautan di *view* dan *controller*.

```
▌ Route diberi nama agar tautan tidak terikat pada URL fisik
Route::get('/menus/{menu}', [TenantMenuController::class, 'show'])
    ->name('tenant.menus.show');
```

```
// Di view atau controller, tautan dibuat dari nama route.
$url = route('tenant.menus.show', [$tenant, $menu]);
```

2.2.3 Middleware memeriksa request

Middleware adalah lapisan yang memeriksa atau memodifikasi *request* sebelum *request* mencapai *controller*. Middleware juga dapat memproses *response* sebelum dikirim kembali kepada pengguna. Beberapa contoh penggunaan middleware adalah sebagai berikut.

- memastikan pengguna sudah login (autentikasi);
- membatasi akses berdasarkan peran (otorisasi awal);
- menetapkan konteks tenant pada aplikasi multi-tenant;
- memvalidasi sesi meja pelanggan;
- membatasi laju request, misalnya pemindaian QR;
- mencatat log dan menambahkan header pada response.

Dengan middleware, pemeriksaan lintas-fitur tidak perlu ditulis berulang pada setiap controller.

2.2.4 Controller mengatur interaksi

Controller menerima *request* yang telah lolos dari *middleware*, lalu mengoordinasikan pekerjaan aplikasi. Controller yang baik tidak menyimpan seluruh logika bisnis secara langsung. Controller sebaiknya menerima dan memvalidasi *request*, memanggil *service* atau *model* yang sesuai, lalu memilih *response* yang akan dikembalikan. Respons dapat berupa halaman HTML, *redirect*, JSON, berkas, atau kode status HTTP tertentu.

2.2.5 View dan response menyampaikan hasil

Jika pengguna membutuhkan halaman HTML, *controller* dapat mengembalikan *view* Blade, misalnya `return view('customer.home', ['canteen' => $canteen]);`. Jika *request* berasal dari API atau interaksi JavaScript, *controller* dapat mengembalikan JSON, misalnya `return response()->json(['data' => $menus]);`. Pemilihan respons mengikuti kebutuhan klien: halaman yang ditampilkan langsung di browser lazim menggunakan HTML, sedangkan antarmuka yang diperbarui melalui JavaScript lazim menggunakan JSON.

```
Dua bentuk respons yang lazim
// Respons HTML: mengembalikan view Blade.
return view('customer.home', ['canteen' => $canteen]);

// Respons JSON: untuk API atau pembaruan antarmuka melalui JavaScript.
return response()->json(['data' => $menus]);
```

2.2.6 Session menyimpan state lintas-request

HTTP bersifat *stateless*: server tidak otomatis mengingat *request* sebelumnya. *Session* digunakan untuk menyimpan data per pengguna lintas-*request*, misalnya pilihan tenant, keranjang sementara, atau pesan keberhasilan setelah *redirect*. Session tidak boleh dijadikan tempat untuk mengabaikan pemeriksaan keamanan. Data yang berasal dari session tetap perlu divalidasi ulang ketika digunakan untuk mengambil data atau memberi akses.

2.3 Modular Monolith dan Batas Domain

Setelah memahami alur *request*, langkah berikutnya adalah menentukan cara menata kode. Pada bab ini, aplikasi Kantin disusun sebagai *modular monolith*.

2.3.1 Memahami modular monolith

Monolith adalah aplikasi yang dijalankan sebagai satu penerapan atau satu unit *deployment*. Pada *modular monolith*, aplikasi tetap dijalankan sebagai satu unit, tetapi kode dibagi menjadi modul dengan tanggung jawab dan batas dependensi yang jelas. Pendekatan ini berbeda dari sistem yang seluruh kodenya tercampur dalam satu struktur global. Pendekatan ini juga berbeda dari *microservices*, karena modul-modulnya belum dijalankan sebagai layanan terpisah.

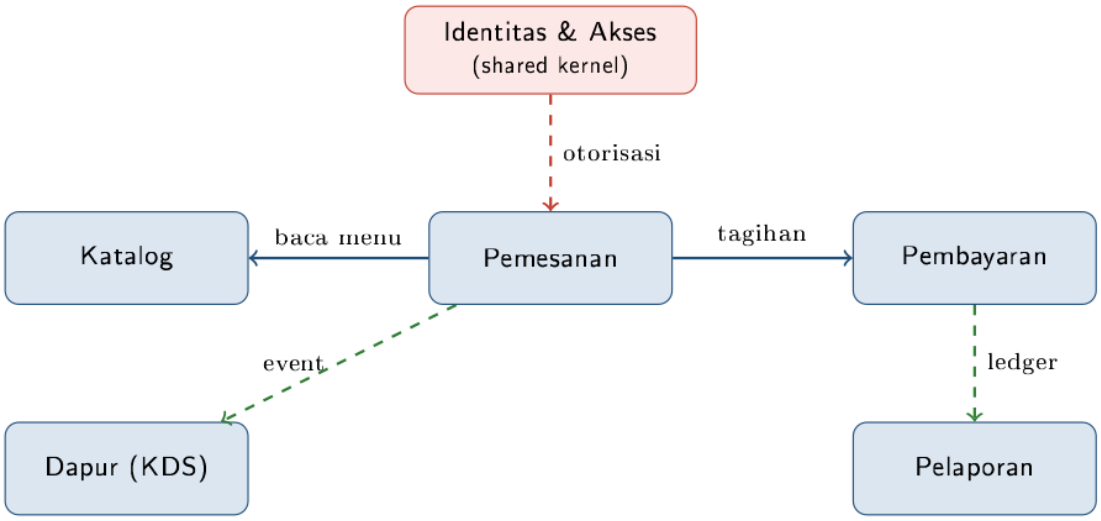
Untuk aplikasi pembelajaran dan aplikasi berskala kecil hingga menengah, *modular monolith* dapat menjadi pilihan yang masuk akal karena:

- pengembangan dan *deployment* lebih sederhana;
- kode tetap dapat dikelompokkan berdasarkan domain;
- pengujian dan penelusuran fitur lebih mudah;
- pemisahan layanan terdistribusi belum diperlukan.

Namun, modularisasi tidak cukup dilakukan dengan membuat banyak folder. Modul harus mempunyai tanggung jawab, batas data, dan arah dependensi yang jelas.

2.3.2 Bounded context dan pemetaan domain

Domain-Driven Design (DDD) menyediakan konsep *bounded context*, yaitu batas eksplisit tempat model domain berlaku secara konsisten. Di dalam satu *bounded context*, istilah, aturan, dan data mempunyai makna yang disepakati. Pada aplikasi Kantin, istilah "pesanan" dapat memiliki penekanan berbeda pada beberapa konteks: dalam konteks Pemesanan, pesanan adalah permintaan pelanggan; dalam konteks Dapur, pesanan dipandang sebagai antrean pekerjaan memasak; dalam konteks Pembayaran, pesanan berkaitan dengan tagihan dan status pelunasan. Peta *bounded context* aplikasi Kantin ditunjukkan pada Gambar 2.2.



Gambar 2.2. Peta bounded context aplikasi Kantin Multi-Tenant.

Sumber: diadaptasi dari SRS Aplikasi Kantin

Tabel 2.1 merangkum tanggung jawab tiap konteks. Pemisahan tersebut membantu menghindari satu model besar yang memuat seluruh kebutuhan aplikasi.

Bounded context	Tanggung jawab utama	Contoh data atau aturan
Identitas dan Akses	Identitas pengguna, autentikasi, dan hak akses	pengguna, peran, status login
Katalog	Penyajian dan pengelolaan menu	menu, kategori, harga, ketersediaan
Pemesanan	Pembentukan dan pelacakan pesanan	keranjang, item pesanan, status pesanan
Pembayaran	Penagihan dan status pembayaran	tagihan, metode pembayaran, komisi
Dapur	Pengelolaan antrean produksi	antrean masak, status diproses, siap
Pelaporan	Penyajian data ringkasan untuk analisis	rekap penjualan, laporan tenant, komisi

Tabel 2.1. Contoh bounded context aplikasi Kantin Multi-Tenant.

Catatan. Identitas dan akses sering dibutuhkan oleh banyak modul. Namun, kebutuhan bersama tidak berarti seluruh model dan data harus dibuka untuk semua modul. Bagikan hanya kontrak atau informasi yang diperlukan. Pada proyek, konteks Identitas dan Akses diwujudkan melalui *middleware* dan *policy*, bukan tabel yang dibuka bebas antar-modul.

2.3.3 Menentukan pemilik tanggung jawab

Setiap kapabilitas bisnis perlu memiliki pemilik yang jelas. Sebagai contoh, perubahan harga menu dimiliki oleh konteks Katalog. Pembuatan pesanan dimiliki oleh konteks Pemesanan. Pengubahan status memasak dimiliki oleh konteks Dapur. Penetapan pemilik mencegah aturan bisnis yang sama ditulis di banyak tempat. Relasi antarkonteks dapat dilakukan melalui pemanggilan langsung atau peristiwa (*event*): pemanggilan langsung ketika satu modul membutuhkan data secara segera, dan peristiwa ketika satu modul perlu memberi tahu modul lain tentang perubahan keadaan. Pemilihan mekanisme mengikuti kebutuhan konsistensi data, kompleksitas aplikasi, dan kemampuan tim.

2.3.4 Separation of concerns

Separation of concerns adalah prinsip untuk membagi sistem agar setiap bagian menangani satu jenis tanggung jawab utama. Dalam aplikasi ini, pembagian sederhana dapat dilakukan seperti pada Tabel 2.2.

Lapisan/komponen	Tanggung jawab utama	Tidak seharusnya menjadi tanggung jawab utama
Route	Memetakan URL dan middleware	Menghitung aturan bisnis kompleks
Middleware	Menangani urusan lintas-request	Menyusun halaman atau menyimpan seluruh logika fitur
Controller	Mengoordinasikan request dan response	Menampung seluruh logika bisnis
Service	Menjalankan logika bisnis	Merender HTML
Model	Merepresentasikan dan mengakses data	Mengatur navigasi antarmuka
View Blade	Menyajikan data	Menetapkan aturan otorisasi utama

Tabel 2.2. Pemisahan tanggung jawab antar-lapisan (data ilustratif).

Pembagian ini tidak berarti setiap aplikasi harus memiliki jumlah lapisan yang sama. Yang penting, setiap keputusan penempatan kode dapat dijelaskan berdasarkan tanggung jawabnya.

2.3.5 Menganalisis kopling modul

Kopling menggambarkan hubungan ketergantungan antar-modul. Modul yang terlalu banyak mengetahui detail modul lain akan lebih sulit diubah. Salah satu metrik yang dapat digunakan untuk mendiskusikan arah dependensi adalah *instability* dari Robert C. Martin.

$$I = \frac{C_e}{C_a + C_e}, \qquad 0 \le I \le 1 \tag{2.1}$$

dengan *C_e* kopling keluar (*efferent*, jumlah modul yang dirujuk oleh suatu modul), *C_a* kopling masuk (*afferent*, jumlah modul yang merujuk pada suatu modul), dan *I* nilai ketidakstabilan pada rentang 0–1 selama *C_a* + *C_e* > 0. Nilai mendekati 0 berarti modul lebih stabil; mendekati 1 berarti modul lebih mudah berubah.

Contoh 2.1. Menghitung ketidakstabilan modul Katalog. Misalkan modul Katalog dirujuk oleh modul Pemesanan dan Dapur, sehingga *C_a* = 2. Modul Katalog hanya merujuk pada modul Identitas dan Akses, sehingga *C_e* = 1. Maka I = 1 / (2 + 1) = 1/3 ≈ 0,33. Nilai 0,33 menunjukkan bahwa modul Katalog relatif stabil dalam skenario ilustratif ini. Kesimpulan tersebut masuk akal apabila banyak bagian aplikasi menggunakan data menu, sedangkan Katalog sendiri tidak bergantung pada terlalu banyak modul lain.

Catatan. Metrik ini adalah alat bantu analisis, bukan ukuran tunggal kualitas desain. Nilai yang baik bergantung pada peran modul, aturan dependensi, dan kebutuhan perubahan sistem. Jangan mengubah desain hanya untuk mengejar angka tertentu. Jika *C_a* = 0 dan *C_e* = 0, Persamaan 2.1 tidak dapat digunakan karena penyebutnya bernilai nol.

2.4 Menyusun Modul dan Service Provider

Konsep domain perlu diterjemahkan menjadi struktur kode yang mudah ditelusuri. Pada proyek Kantin, organisasi kode bersifat *modular monolith* dengan pembagian tanggung jawab yang konsisten.

2.4.1 Organisasi direktori berbasis domain

Struktur proyek memisahkan logika domain ke dalam modul, sedangkan *controller* dikelompokkan berdasarkan konteks pengguna dan *view* diletakkan pada direktori tampilan bersama. Struktur berikut menggambarkan penataan tersebut.

```
app/
├─ Modules/                (logika domain per bounded context)
│   ├─ Catalog/
│   │   ├─ CatalogServiceProvider.php
│   │   └─ Services/          (MenuStockService, PublicCatalogQuery)
│   ├─ Ordering/  Payments/  Kitchen/  Admin/  Reporting/
├─ Http/
│   ├─ Controllers/         (dikelompokkan per konteks pengguna)
│   │   ├─ Customer/  Tenant/  Admin/  Webhooks/
│   └─ Middleware/          (SetTenantContext, EnsureUserHasRole, ...)
├─ Models/                  (Eloquent: Menu, Tenant, Order, ...)
resources/views/            (Blade: customer/, tenant/, admin/, components/)
routes/                     (web.php, customer.php, tenant.php, admin.php)
```

Struktur direktori proyek Kantin (ringkas).

Tabel 2.3 merangkum tanggung jawab tiap bagian.

Komponen	Tanggung jawab
app/Modules/<Konteks>/Services	Logika bisnis domain yang dapat diuji terpisah
app/Modules/<Konteks>/…ServiceProvider	Titik perakitan modul: binding, event, policy
app/Http/Controllers/<Konteks>	Menerima request, memanggil service, mengembalikan response
app/Http/Middleware	Pemeriksaan lintas-request (tenant, peran, laju)
app/Models	Representasi data dan relasi melalui Eloquent
resources/views	Template Blade dan komponen tampilan
routes/*.php	Definisi route per konteks pengguna

Tabel 2.3. Komponen proyek dan tanggung jawabnya.

Tip. Gunakan modul ketika suatu fitur memiliki tanggung jawab bisnis yang jelas dan berpotensi berkembang. Jangan membuat modul hanya untuk satu berkas kecil yang tidak memiliki batas domain bermakna.

2.4.2 Service provider sebagai titik perakitan modul

Service provider adalah tempat untuk mendaftarkan layanan dan konfigurasi aplikasi. Dalam rancangan modular, setiap *bounded context* memiliki satu *service provider*. Pada proyek Kantin, provider modul bersifat ringkas: *binding* kontrak pada `register()`, serta *event* dan *policy* pada `boot()`.

app/Modules/Catalog/CatalogServiceProvider.php

```php
<?php

namespace App\Modules\Catalog;

use Illuminate\Support\ServiceProvider;

final class CatalogServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // Binding kontrak -> implementasi (mis. saat modul diimplementasikan, Modul 7).
    }

    public function boot(): void
    {
        // Pendaftaran event listener dan policy modul.
    }
}
```

Kode 2.1. Kerangka *service provider* modul Catalog. Provider ini didaftarkan pada berkas `bootstrap/providers.php`, tempat seluruh provider modul dirakit (satu provider per *bounded context*). Berbeda dengan pola lama yang memuat rute di dalam provider, sejak Laravel 12 pendaftaran provider terpusat pada berkas tersebut, dan berkas *route* diatur melalui konfigurasi `bootstrap/app.php` (dibahas pada Subbab 2.5).

Dependency injection (DI) adalah teknik menyediakan dependensi dari luar objek. Laravel menggunakan *service container* untuk membantu proses tersebut. Sebagai contoh, sebuah

controller tidak perlu membuat objek layanan katalog secara manual; *controller* dapat memintanya melalui *constructor*.

```
Contoh injeksi melalui constructor
public function __construct(
    private PublicCatalogQuery $catalog,
) {
}
```

Container akan menyediakan objek `PublicCatalogQuery` sesuai *binding* atau kemampuan resolusi kelas yang tersedia. Pendekatan ini membuat kode lebih mudah diuji karena dependensi dapat diganti dengan implementasi lain atau objek tiruan saat pengujian.

2.4.3 Batas dependency injection

DI membuat kode lebih mudah diuji dan lebih mudah diganti implementasinya. Namun, DI bukan alasan untuk menyuntikkan terlalu banyak objek ke setiap *controller*. Jika sebuah *controller* membutuhkan banyak *service* yang tidak berkaitan, periksa kembali kemungkinan berikut: fitur *controller* sebenarnya terlalu luas; beberapa logika perlu dipisahkan menjadi *use case*; batas modul belum cukup jelas; atau *service* memiliki tanggung jawab terlalu besar. Gunakan *dependency injection* untuk memperjelas kebutuhan suatu kelas, bukan untuk menyembunyikan kompleksitas desain.

2.5 Routing dan Controller

Routing dan *controller* merupakan pintu masuk utama interaksi pengguna dengan aplikasi. *Route* menentukan tindakan yang dipanggil, sedangkan *controller* mengoordinasikan proses untuk menghasilkan respons. Karena seluruh permintaan pengguna melewati lapisan ini, kekeliruan kecil pada definisi route dapat berdampak pada keterbacaan URL, konsistensi tautan, hingga terbukanya akses yang seharusnya dibatasi.

Subbab ini membahas routing secara bertahap. Pola dasar diperkenalkan lebih dahulu melalui contoh umum, misalnya blog dan toko daring, agar mekanismenya terlihat jelas tanpa terikat pada satu aplikasi tertentu. Setelah pola dasar dipahami, pembahasan dilanjutkan pada penerapannya di proyek Kantin Multi-Tenant yang menuntut pemisahan konteks pengguna serta isolasi data antar-tenant.

2.5.1 Berkas route, metode HTTP, dan bentuk route sederhana

Definisi route disimpan pada direktori `routes/`. Berkas `routes/web.php` dimuat otomatis melalui konfigurasi `bootstrap/app.php` dan memperoleh kelompok middleware `web` yang menyediakan session, cookie, serta perlindungan CSRF. Untuk antarmuka program yang bersifat *stateless*, berkas `routes/api.php` ditambahkan melalui perintah `php artisan install:api`; berkas tersebut otomatis memperoleh awalan `/api` dan kelompok middleware `api`.

Bentuk route paling sederhana memetakan sebuah URL ke *closure*. Untuk fitur yang sesungguhnya, pemetaan ke metode *controller* lebih dianjurkan karena logika penanganan request menjadi lebih mudah ditelusuri, diuji, dan digunakan ulang.

```
routes/web.php  (dua bentuk route dasar)
use App\Http\Controllers\ArticleController;
use Illuminate\Support\Facades\Route;

// Bentuk closure: cocok untuk contoh singkat atau halaman sangat sederhana.
```

```php
Route::get('/salam', function () {
    return 'Halo, dunia';
});

// Bentuk controller: dianjurkan untuk fitur yang sesungguhnya.
Route::get('/artikel', [ArticleController::class, 'index']);
```

Kode 2.2. Dua bentuk route dasar. Argumen pertama adalah pola URL, argumen kedua adalah tindakan yang dijalankan. Notasi `[Kelas::class, 'metode']` menunjuk metode controller yang menangani request tersebut.

Laravel menyediakan satu metode untuk setiap verba HTTP. Tabel 2.4 merangkum metode yang tersedia beserta kegunaannya yang lazim.

Metode	Sintaks	Kegunaan lazim
GET	Route::get($uri, $aksi)	Mengambil dan menampilkan sumber daya
POST	Route::post($uri, $aksi)	Membuat sumber daya baru
PUT	Route::put($uri, $aksi)	Mengganti seluruh isi sumber daya
PATCH	Route::patch($uri, $aksi)	Memperbarui sebagian isi sumber daya
DELETE	Route::delete($uri, $aksi)	Menghapus sumber daya
OPTIONS	Route::options($uri, $aksi)	Menyatakan metode yang didukung
Beberapa verba	Route::match(['get', 'post'], ...)	Satu URL untuk beberapa verba tertentu
Semua verba	Route::any($uri, $aksi)	Semua verba sekaligus; gunakan secara terbatas

Tabel 2.4. Metode route untuk setiap verba HTTP.

> **Peringatan.** Seluruh route `POST`, `PUT`, `PATCH`, dan `DELETE` pada berkas route web wajib menyertakan token CSRF melalui direktif `@csrf` pada formulir. Tanpa token tersebut, Laravel menolak request dengan kode status 419. Pembahasan formulir dan CSRF dilanjutkan pada Subbab 2.6.

Untuk kebutuhan sederhana, Laravel menyediakan dua route pintasan. Route *redirect* mengalihkan satu URL ke URL lain, sedangkan route *view* langsung mengembalikan sebuah view tanpa perlu membuat controller terlebih dahulu.

```php
Route pintasan: redirect dan view
// Pengalihan sementara (302) dan permanen (301).
Route::redirect('/lama', '/baru');
Route::redirect('/lama', '/baru', 301);
Route::permanentRedirect('/lama', '/baru');

// Langsung mengembalikan view, dengan atau tanpa data.
Route::view('/tentang', 'tentang');
Route::view('/tentang', 'tentang', ['judul' => 'Tentang Kami']);
```

Kode 2.3. Route pintasan. Gunakan `Route::redirect()` ketika alamat lama masih ditautkan dari luar aplikasi, dan `Route::view()` untuk halaman statis yang tidak memerlukan pengolahan data.

2.5.2 Parameter route dan batasannya

Parameter route adalah bagian URL yang nilainya berubah sesuai sumber daya yang diakses. Parameter ditulis di dalam kurung kurawal dan diteruskan ke closure atau metode controller berdasarkan urutan kemunculannya pada URL, bukan berdasarkan kesamaan nama argumen. Parameter dapat dijadikan opsional dengan menambahkan tanda tanya pada pola URL dan memberi nilai bawaan pada argumen penerimanya.

Parameter wajib, ganda, dan opsional

```php
// Parameter wajib.
Route::get('/pengguna/{id}', function (string $id) {
    return 'Pengguna '.$id;
});

// Beberapa parameter sekaligus; urutan argumen mengikuti urutan pada URL.
Route::get('/artikel/{artikel}/komentar/{komentar}', function (
    string $artikel,
    string $komentar,
) {
    // ...
});

// Parameter opsional: URL /profil dan /profil/rina sama-sama cocok.
Route::get('/profil/{nama?}', function (?string $nama = 'tamu') {
    return $nama;
});
```

Kode 2.4. Tiga bentuk parameter route. Objek `Illuminate\Http\Request` dapat ditambahkan sebagai argumen pertama; Laravel menyuntikkannya melalui *service container*, lalu parameter route menyusul sesuai urutan pada URL.

Parameter dapat dibatasi polanya agar route hanya cocok untuk nilai tertentu. Pembatasan dilakukan melalui metode `where()` atau melalui metode pembantu yang lebih ringkas.

Batasan parameter dengan where() dan metode pembantu

```php
// Ekspresi reguler eksplisit.
Route::get('/profil/{nama}', function (string $nama) {
    // ...
})->where('nama', '[A-Za-z]+');

// Beberapa batasan sekaligus.
Route::get('/artikel/{id}/{slug}', function (string $id, string $slug) {
    // ...
})->where(['id' => '[0-9]+', 'slug' => '[a-z0-9-]+']);

// Metode pembantu yang lebih terbaca.
Route::get('/artikel/{id}/{slug}', function (string $id, string $slug) {
    // ...
})->whereNumber('id')->whereAlphaNumeric('slug');

// Membatasi pada sekumpulan nilai yang diizinkan.
Route::get('/kategori/{kategori}', function (string $kategori) {
    // ...
})->whereIn('kategori', ['makanan', 'minuman', 'paket']);
```

Kode 2.5. Pembatasan parameter route. Jika nilai pada URL tidak memenuhi pola, route dianggap tidak cocok dan aplikasi mengembalikan respons 404.

Metode pembantu yang tersedia dirangkum pada Tabel 2.5.

Metode pembantu	Pola yang diizinkan
whereNumber('id')	Angka saja
whereAlpha('nama')	Huruf saja
whereAlphaNumeric('slug')	Huruf dan angka
whereUuid('id')	Format UUID
whereUlid('id')	Format ULID
whereIn('kategori', [...])	Salah satu nilai pada daftar atau enum
where('nama', '[A-Za-z]+')	Ekspresi reguler bebas

Tabel 2.5. Metode pembantu untuk membatasi parameter route.

Jika sebuah pola berlaku untuk seluruh aplikasi, batasan dapat didaftarkan sekali melalui `Route::pattern()` pada metode `boot()` di `AppServiceProvider`, misalnya `Route::pattern('id', '[0-9]+');`. Secara bawaan parameter route tidak menerima karakter garis miring; jika karakter tersebut memang diperlukan, gunakan `->where('search', '.*')` pada parameter terakhir.

> **Catatan.** Batasan parameter bukan pengganti validasi. Nilai yang tidak cocok menghasilkan respons 404, bukan pesan kesalahan yang dapat dibaca pengguna. Gunakan batasan untuk menjaga bentuk URL, dan gunakan validasi di controller atau *form request* untuk memeriksa kelayakan data.

2.5.3 Nama route dan pembuatan URL

Nama route memberi identitas tetap pada sebuah definisi route. Dengan identitas tersebut, tautan pada view maupun pengalihan pada controller dibuat berdasarkan nama, bukan berdasarkan URL fisik. Ketika struktur URL berubah, cukup definisi route yang diperbarui; seluruh tautan mengikuti secara otomatis.

```
Penamaan route dan pembuatan URL
Route::get('/pengguna/profil', [ProfileController::class, 'show'])
    ->name('profil');

Route::get('/artikel/{id}/komentar', [CommentController::class, 'index'])
    ->name('artikel.komentar');

$url = route('profil');

// Parameter yang tidak dikenali route menjadi query string.
$url = route('artikel.komentar', ['id' => 7, 'halaman' => 2]);
// hasil: /artikel/7/komentar?halaman=2

// Pengalihan ke route bernama.
return redirect()->route('profil');
return to_route('profil');
```

Kode 2.6. Nama route dan helper `route()`. Konvensi penamaan bertitik, misalnya `tenant.menus.index`, membentuk hierarki yang mudah dibaca dan mudah dicari.

Nama route juga dapat diperiksa saat aplikasi berjalan. Metode `Route::currentRouteName()` mengembalikan nama route yang sedang diproses, sedangkan `$request->route()-`

`>named('profil')` berguna di dalam middleware untuk memberi perlakuan khusus pada route tertentu. Kemampuan ini sering dipakai untuk menandai butir navigasi yang sedang aktif.

> **Tip.** Perlakukan nama route sebagai kontrak antar-lapisan. View, controller, dan pengujian sebaiknya menyebut nama route, bukan URL. Kebiasaan ini membuat perubahan struktur URL menjadi pekerjaan pada satu berkas, bukan pekerjaan pada seluruh proyek.

2.5.4 Route group: awalan, nama, middleware, controller, dan subdomain

Route yang memiliki sifat sama sebaiknya dikelompokkan. *Route group* memungkinkan awalan URL, awalan nama, daftar middleware, controller, hingga subdomain ditetapkan satu kali untuk banyak route sekaligus. Pengelompokan ini mengurangi pengulangan sekaligus memperjelas batas akses tiap kelompok fitur.

Bentuk-bentuk route group

```php
// Awalan URL.
Route::prefix('admin')->group(function () {
    Route::get('/pengguna', [UserController::class, 'index']);   // /admin/pengguna
});

// Awalan nama route.
Route::name('admin.')->group(function () {
    Route::get('/pengguna', [UserController::class, 'index'])
        ->name('pengguna');                                      // admin.pengguna
});

// Middleware bersama.
Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('/dasbor', [DashboardController::class, 'index']);
});

// Controller bersama: cukup menyebut nama metode.
Route::controller(OrderController::class)->group(function () {
    Route::get('/pesanan/{id}', 'show');
    Route::post('/pesanan', 'store');
});

// Subdomain: nilai subdomain menjadi argumen pertama.
Route::domain('{akun}.example.com')->group(function () {
    Route::get('/pengguna/{id}', function (string $akun, string $id) {
        // ...
    });
});
```

Kode 2.7. Lima bentuk route group. Seluruh bentuk dapat dirantai, misalnya `Route::middleware('auth')->prefix('admin')->name('admin.')->group(...)`. Awalan URL, awalan nama, dan daftar middleware bersifat kumulatif ketika grup disarangkan.

Pada proyek Kantin, ketiga kelompok pengguna dipisahkan dengan menggabungkan awalan URL, awalan nama, dan middleware. Pengelompokan tersebut dikonfigurasi pada berkas `bootstrap/app.php` sehingga setiap berkas route tetap ringkas dan hanya berisi definisi fitur.

bootstrap/app.php (kutipan withRouting)

```php
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        health: '/up',
        then: function (): void {
            // Konteks PELANGGAN (publik, anonim).
            Route::middleware('web')
                ->prefix('kantin/{canteen}')
```

```
                ->name('customer.')
                ->group(base_path('routes/customer.php'));

            // Konteks OPERATOR TENANT (internal): auth + verified + tenant.
            Route::middleware(['web', 'auth', 'verified', 'tenant'])
                ->prefix('tenant/{tenant:slug}')
                ->name('tenant.')
                ->scopeBindings()
                ->group(base_path('routes/tenant.php'));

            // Konteks PENGELOLA KANTIN (internal): auth + verified + role:admin.
            Route::middleware(['web', 'auth', 'verified', 'role:admin'])
                ->prefix('admin')
                ->name('admin.')
                ->group(base_path('routes/admin.php'));
        },
    )
```

Kode 2.8. Konfigurasi *route group* berdasarkan konteks pengguna. Alias *middleware* `tenant` dan `role` didaftarkan pada berkas yang sama (Subbab 2.7). Perhatikan tiga hal: kelompok pelanggan hanya memakai *middleware* `web`; kelompok tenant menambahkan `auth`, `verified`, `tenant`, serta `scopeBindings()` untuk mengunci sumber daya di bawah tenant induk; kelompok admin membatasi akses dengan `role:admin`.

Karena awalan dan nama *route* sudah ditetapkan pada tingkat grup, berkas *route* cukup memuat definisi ringkas. Kode 2.9 menunjukkan berkas *route* modul tenant.

routes/tenant.php (kutipan)

```
use App\Http\Controllers\Tenant\TenantMenuController;
use App\Models\Tenant;
use Illuminate\Support\Facades\Route;

Route::get('/menus', [TenantMenuController::class, 'index'])
    ->name('menus.index');

Route::get('/menus/{menu}', [TenantMenuController::class, 'show'])
    ->name('menus.show');

Route::patch('/menus/{menu}', [TenantMenuController::class, 'update'])
    ->name('menus.update');
```

Kode 2.9. Berkas *route* konteks tenant. Karena grup memberi awalan `tenant/{tenant:slug}` dan nama `tenant.`, *route* pertama menghasilkan URL `tenant/{slug}/menus` dengan nama lengkap `tenant.menus.index`. Nama *route* dipakai oleh *view* dan *controller* agar tautan tidak bergantung langsung pada URL fisik.

2.5.5 Route model binding

Sebagian besar parameter route sesungguhnya menunjuk sebuah baris data. *Route model binding* membuat Laravel mengubah nilai parameter menjadi objek model secara otomatis, sehingga metode controller langsung menerima objek yang siap dipakai. Pendekatan ini menghilangkan kode pencarian manual seperti `Menu::findOrFail($id)` dan secara bawaan menghasilkan respons 404 ketika data tidak ditemukan.

Pengikatan implisit terjadi ketika nama parameter route sama dengan nama argumen yang bertipe model. Secara bawaan Laravel mencari berdasarkan kolom kunci utama; kolom lain dapat digunakan dengan menuliskannya setelah tanda titik dua pada pola URL.

Pengikatan implisit dan kunci kustom

```php
use App\Models\Post;

// Nama parameter {post} sama dengan nama argumen $post bertipe Post.
Route::get('/artikel/{post}', function (Post $post) {
    return $post->title;
});

// Bentuk yang sama pada controller.
Route::get('/artikel/{post}', [PostController::class, 'show']);

// Mengikat berdasarkan kolom slug, bukan id.
Route::get('/artikel/{post:slug}', function (Post $post) {
    return $post->title;
});
```

Kode 2.10. Pengikatan implisit. URL `/artikel/rahasia-kopi` pada contoh terakhir dicari melalui kolom `slug`, sehingga URL menjadi lebih mudah dibaca sekaligus tidak menampakkan urutan identitas pada basis data.

Jika sebuah model selalu diikat melalui kolom yang sama, kolom tersebut dapat ditetapkan pada modelnya. Selain model Eloquent, Laravel juga dapat mengikat parameter menjadi *enum*, menyertakan data yang dihapus sementara, serta mengganti perilaku ketika data tidak ditemukan.

Kunci bawaan model, enum, data terhapus, dan penanganan data tak ditemukan

```php
use Illuminate\Database\Eloquent\Attributes\RouteKey;

#[RouteKey('slug')]
class Post extends Model
{
    // Seluruh route yang mengikat Post kini memakai kolom slug.
}

enum Kategori: string
{
    case Makanan = 'makanan';
    case Minuman = 'minuman';
}

// Hanya nilai yang ada pada enum yang diterima; selain itu 404.
Route::get('/kategori/{kategori}', function (Kategori $kategori) {
    return $kategori->value;
});

// Menyertakan data yang dihapus sementara (soft delete).
Route::get('/artikel/{post}', [PostController::class, 'show'])->withTrashed();

// Mengganti respons 404 dengan pengalihan.
Route::get('/lokasi/{lokasi:slug}', [LocationController::class, 'show'])
    ->name('lokasi.show')
    ->missing(fn (Request $request) => redirect()->route('lokasi.index'));
```

Kode 2.11. Variasi pengikatan model. Atribut `#[RouteKey]` menetapkan kolom pengikatan pada tingkat model, sedangkan `missing()` berguna ketika pengalaman pengguna lebih baik diarahkan ke halaman daftar daripada ke halaman kesalahan.

Ketika sebuah sumber daya berada di bawah sumber daya lain, pengikatan perlu dibatasi pada relasi induknya. Tanpa pembatasan tersebut, artikel milik pengguna lain masih dapat ditemukan

melalui URL yang seolah-olah berada di bawah pengguna tertentu. Laravel menyebut mekanisme ini *scoped binding*.

```
Pembatasan pengikatan bersarang (scoped bindings)
// Kunci kustom pada parameter anak: pencarian otomatis dibatasi relasi induk.
Route::get('/pengguna/{user}/artikel/{post:slug}', function (
    User $user,
    Post $post,
) {
    return $post;
});

// Tanpa kunci kustom, pembatasan relasi diminta secara eksplisit.
Route::get('/pengguna/{user}/artikel/{post}', function (
    User $user,
    Post $post,
) {
    return $post;
})->scopeBindings();

// Berlaku untuk seluruh route di dalam grup.
Route::scopeBindings()->group(function () {
    Route::get('/pengguna/{user}/artikel/{post}', function (
        User $user,
        Post $post,
    ) {
        return $post;
    });
});
```

Kode 2.12. Pembatasan pengikatan bersarang. Laravel mencari artikel melalui relasi `$user->posts()`, bukan melalui seluruh tabel artikel. Pembatasan dapat dimatikan kembali dengan `withoutScopedBindings()` apabila memang diperlukan.

Apabila aturan pencarian tidak dapat dinyatakan cukup melalui nama kolom, pengikatan dapat didefinisikan sendiri melalui *pengikatan eksplisit* pada `AppServiceProvider`.

```
Pengikatan eksplisit
// Pada metode boot() AppServiceProvider.
Route::model('user', User::class);

Route::bind('user', function (string $value) {
    return User::where('username', $value)->firstOrFail();
});
```

Kode 2.13. Pengikatan eksplisit. Alternatif lain adalah menimpa metode `resolveRouteBinding()` pada model yang bersangkutan.

Pada proyek Kantin, ketiga mekanisme tersebut dipakai bersama-sama. Parameter `{tenant:slug}` memakai kunci kustom agar URL mudah dibaca, `scopeBindings()` pada tingkat grup mengunci `{menu}` di bawah tenant induknya, sedangkan *global scope* pada model menyaring data ke tenant yang sedang aktif.

> **Peringatan.** Pada aplikasi multi-tenant, menemukan menu berdasarkan ID belum cukup untuk memastikan menu tersebut milik tenant yang sedang aktif. Pada proyek Kantin, isolasi ditegakkan berlapis: *global scope* memfilter data ke tenant aktif, `scopeBindings()` mengunci `{menu}` di bawah `{tenant}`, dan *policy* memeriksa hak akses per aksi. Pembahasan pembatasan data tenant diperdalam pada Bab 4.

2.5.6 Fallback, pembatasan laju, dan spoofing metode formulir

Route *fallback* menangani request yang tidak cocok dengan route mana pun. Route ini harus didefinisikan paling akhir karena Laravel mencocokkan route secara berurutan.

Pembatasan laju melindungi aplikasi dari permintaan yang terlalu sering, baik karena kesalahan klien maupun karena percobaan penyalahgunaan. Aturan pembatasan didaftarkan melalui `RateLimiter::for()`, lalu dipasang pada route melalui middleware `throttle`.

Route fallback dan pembatasan laju

```php
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;

// Didaftarkan pada metode boot() AppServiceProvider.
RateLimiter::for('pemindaian-qr', function (Request $request) {
    return Limit::perMinute(20)->by($request->ip());
});

RateLimiter::for('masuk', function (Request $request) {
    return [
        Limit::perMinute(500),
        Limit::perMinute(3)->by($request->input('email')),
    ];
});

// Dipasang pada route melalui middleware throttle.
Route::middleware('throttle:pemindaian-qr')->group(function () {
    Route::post('/meja/pindai', [TableScanController::class, 'store']);
});

// Didefinisikan paling akhir.
Route::fallback(function () {
    return response()->view('errors.404', [], 404);
});
```

Kode 2.14. Pembatasan laju dan route fallback. Metode `by()` menentukan dasar penghitungan, misalnya alamat IP atau identitas pengguna. Ketika batas terlampaui, Laravel mengembalikan kode status 429.

Formulir HTML hanya mendukung metode GET dan POST. Agar route PUT, PATCH, atau DELETE tetap dapat dipanggil dari formulir, Laravel membaca medan tersembunyi `_method` yang dihasilkan direktif `@method`. Teknik ini disebut *form method spoofing* dan contoh penerapannya dapat dilihat pada Kode 2.19.

2.5.7 Controller

Controller sebaiknya bertindak sebagai pengatur alur, bukan sebagai tempat seluruh aturan bisnis. Controller menerima input, memanggil *service* atau model, lalu mengembalikan respons. Kode 2.15 memperlihatkan *controller* menu tenant pada proyek.

app/Http/Controllers/Tenant/TenantMenuController.php

```php
<?php

namespace App\Http\Controllers\Tenant;

use App\Http\Controllers\Controller;
use App\Models\Menu;
use App\Models\Tenant;
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Http\JsonResponse;
```

```php
use Illuminate\Http\Request;

final class TenantMenuController extends Controller
{
    use AuthorizesRequests;

    public function show(Tenant $tenant, Menu $menu): JsonResponse
    {
        // Otorisasi per aksi (policy) di server, bukan sekadar navigasi.
        $this->authorize('view', $menu);

        return response()->json(
            $menu->only(['id', 'name', 'base_price', 'is_available']),
        );
    }

    public function update(Request $request, Tenant $tenant, Menu $menu): JsonResponse
    {
        $this->authorize('update', $menu);

        // Validasi server tetap wajib meski formulir sudah memberi batas di browser.
        $data = $request->validate(['is_available' => ['required', 'boolean']]);
        $menu->update(['is_available' => $data['is_available']]);

        return response()->json(['ok' => true, 'is_available' => $menu->is_available]);
    }
}
```

Kode 2.15. *Controller* menu tenant. Alur kodenya: *route* mengarahkan *request* ke metode; Laravel mengikat parameter {menu} menjadi objek Menu; *controller* memeriksa otorisasi melalui *policy*, memvalidasi input, lalu mengembalikan respons. Controller tetap ringkas karena pemeriksaan akses berada pada *policy* dan isolasi data berada pada *global scope* model.

2.5.8 Resource controller untuk CRUD

Laravel menyediakan *resource controller* untuk pola operasi CRUD yang umum. CRUD merupakan singkatan dari *create*, *read*, *update*, dan *delete*. Tabel 2.6 menampilkan konvensi pemetaannya.

Metode HTTP	URL	Aksi controller	Kegunaan
GET	/menus	index	Menampilkan daftar menu
GET	/menus/create	create	Menampilkan formulir tambah
POST	/menus	store	Menyimpan menu baru
GET	/menus/{menu}	show	Menampilkan satu menu
GET	/menus/{menu}/edit	edit	Menampilkan formulir ubah
PUT atau PATCH	/menus/{menu}	update	Memperbarui menu
DELETE	/menus/{menu}	destroy	Menghapus menu

Tabel 2.6. Konvensi resource controller.

Jika seluruh tindakan dibutuhkan, *route* dapat didefinisikan dengan pola `Route::resource('menus', TenantMenuController::class);`. Jika hanya beberapa tindakan diperlukan, batasi dengan `->only([...])`. Pada proyek Kantin, konteks tenant hanya mendefinisikan `index`, `show`, dan `update` secara eksplisit karena penambahan menu dilakukan

melalui antarmuka Livewire (dibahas pada Modul 7). Pilih *resource controller* ketika pola fitur benar-benar mengikuti operasi CRUD; jangan memaksakannya untuk proses bisnis yang memiliki alur khusus, misalnya “konfirmasi pembayaran” atau “tandai pesanan siap”.

2.5.9 Menelusuri dan menyimpan cache route

Ketika jumlah route bertambah, daftar route menjadi dokumentasi hidup bagi tim pengembang. Perintah `route:list` menampilkan seluruh route beserta metode, URL, nama, dan aksi yang menanganinya. Perintah ini juga merupakan cara tercepat memeriksa apakah awalan dan middleware sebuah grup sudah sesuai rancangan.

```
Perintah artisan untuk route
# Menampilkan seluruh route.
php artisan route:list

# Menampilkan middleware setiap route.
php artisan route:list -v

# Menyaring berdasarkan URL dan mengabaikan route milik paket pihak ketiga.
php artisan route:list --path=tenant --except-vendor

# Menyimpan dan menghapus cache route (dijalankan saat penerapan ke produksi).
php artisan route:cache
php artisan route:clear
```

Kode 2.16. Perintah artisan yang berkaitan dengan route. Cache route mempercepat pendaftaran route pada lingkungan produksi dan harus dibuat ulang setiap kali definisi route berubah.

> **Peringatan.** Route berbentuk *closure* tidak dapat disimpan pada cache route. Inilah salah satu alasan praktis untuk memakai controller pada fitur yang sesungguhnya dan membatasi closure pada contoh pembelajaran.

2.6 Lapisan Presentasi dengan Blade

Lapisan presentasi bertugas menyajikan data kepada pengguna. Pada Laravel, lapisan ini lazim dibangun menggunakan *Blade Template*.

2.6.1 Peran view

View menerima data yang telah disiapkan *controller* atau *service*, lalu menampilkannya dalam bentuk HTML. *View* tidak seharusnya menjadi tempat utama untuk menghitung aturan bisnis, melakukan *query* basis data, atau menegakkan otorisasi. Pemisahan tersebut membantu Anda melakukan perubahan tampilan tanpa harus mengubah aturan bisnis.

2.6.2 Blade dan escaping output

Blade adalah mesin *template* Laravel yang dikompilasi menjadi PHP. Blade menyediakan direktif untuk menampilkan data, membuat percabangan, melakukan perulangan, memakai layout, membuat komponen, serta menampilkan pesan validasi. Tabel 2.7 merangkum direktif yang sering digunakan.

Direktif	Fungsi
{{ $x }}	Menampilkan data dengan escaping HTML
@if, @elseif, @else	Percabangan tampilan

@foreach, @forelse	Perulangan data
@extends, @section, @yield	Pewarisan layout klasik
@props, {{ $slot }}	Mendefinisikan komponen dan slotnya
@include	Menyisipkan sub-view
@csrf	Menambahkan token CSRF pada formulir
@method	Menambahkan spoofing metode HTTP pada formulir
@error	Menampilkan pesan kesalahan validasi

Tabel 2.7. Direktif Blade yang sering digunakan.

Sintaks `{{ $x }}` membantu melakukan *escaping* HTML pada nilai yang ditampilkan. Hal ini penting untuk mengurangi risiko *Cross-Site Scripting* (XSS) ketika teks berasal dari pengguna atau sumber yang tidak sepenuhnya tepercaya. Namun, *escaping* output bukan satu-satunya pertahanan keamanan aplikasi. Anda tetap perlu berhati-hati saat menangani HTML mentah, URL, atribut HTML, kode JavaScript, dan data dari sumber eksternal.

2.6.3 Menampilkan daftar menu

Kode 2.17 menampilkan daftar menu menggunakan Blade beserta komponen status.

resources/views/customer/menu-list.blade.php (ilustratif)

```
<x-layouts.customer :title="'Katalog Menu'">
    <h1>Menu Tenant</h1>

    @forelse ($menus as $menu)
        <div class="menu-item">
            <h2>{{ $menu->name }}</h2>
            <p>Rp{{ number_format($menu->base_price, 0, ',', '.') }}</p>

            <x-status-badge :status="$menu->is_available ? 'active' : 'suspended'">
                {{ $menu->is_available ? 'Tersedia' : 'Habis' }}
            </x-status-badge>
        </div>
    @empty
        <p>Belum ada menu tersedia.</p>
    @endforelse
</x-layouts.customer>
```

Pada contoh tersebut: `@forelse` menampilkan setiap menu apabila koleksi berisi data, sedangkan bagian `@empty` ditampilkan jika tidak ada menu; `{{ $menu->name }}` menampilkan nama menu dengan *escaping* HTML; dan komponen `<x-status-badge>` menampilkan status. Pemanggilan `number_format()` masih dapat digunakan untuk contoh sederhana. Namun, jika format harga dipakai berulang pada banyak halaman, pertimbangkan untuk memusatkan format pada *helper*, *presenter*, atau komponen agar *view* tidak penuh oleh detail format.

2.6.4 Blade component

Komponen Blade adalah unit tampilan yang dapat dipakai ulang. Komponen berguna ketika markup dan perilaku tampilan yang sama muncul pada banyak halaman. Kode 2.6 menampilkan komponen status pada proyek.

resources/views/components/status-badge.blade.php

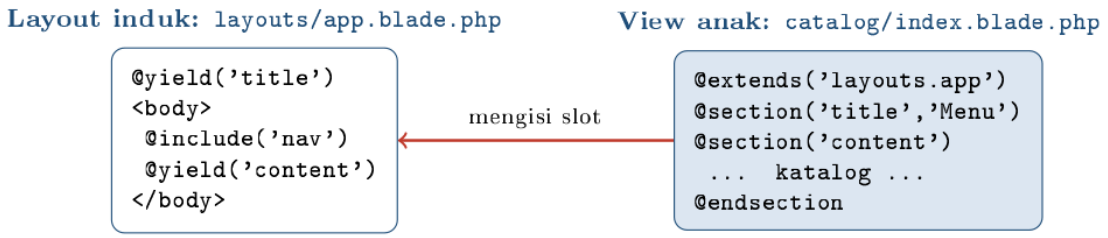
```
@props(['status' => 'default'])
@php
$map = [
```

```
    'active'    => 'bg-green-100 text-green-800',
    'pending'   => 'bg-amber-100 text-amber-800',
    'suspended' => 'bg-red-100 text-red-800',
    'default'   => 'bg-zinc-100 text-zinc-700',
];
@endphp
<span {{ $attributes->class([
    'inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-medium',
    $map[$status] ?? $map['default'],
]) }}>{{ $slot }}</span>
```

Kode 2.18. Komponen `status-badge` pada proyek. Komponen menerima properti `status` melalui `@props`, memetakannya ke kelas gaya, lalu menampilkan isi melalui `{{ $slot }}`. Dipanggil dari *view* lain sebagai `<x-status-badge status="active">Aktif</x-status-badge>`. Jika label atau warna status diubah, Anda cukup mengubah satu komponen. Gunakan komponen untuk bagian antarmuka yang berulang dan bermakna; hindari membuat komponen yang terlalu kecil jika penggunaannya hanya sekali.

2.6.5 Layout berlapis

Layout adalah kerangka halaman yang berisi bagian bersama, misalnya `<head>`, *header*, navigasi, area konten, dan *footer*. Blade klasik menyediakan pewarisan layout melalui `@extends` untuk menunjuk layout induk, `@section` untuk mengisi bagian tertentu, dan `@yield` untuk menyediakan slot pada layout induk. Gambar 2.3 mengilustrasikan pewarisan tersebut.



Gambar 2.3. Pewarisan layout Blade: view anak mengisi slot layout induk.

Sumber: diadaptasi penulis (ilustrasi konseptual).

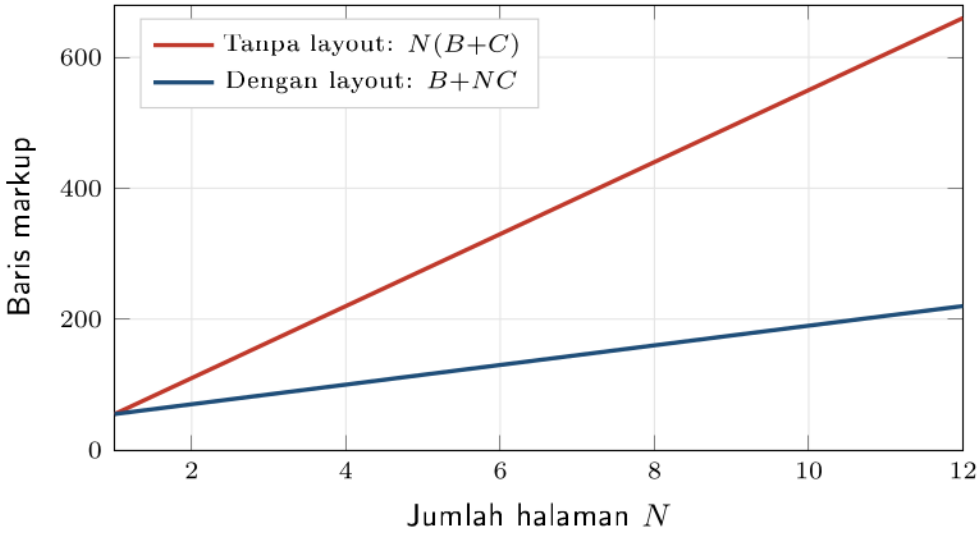
Catatan. Proyek Kantin menggunakan pendekatan layout berbasis komponen (*component-based layout*), yaitu `<x-layouts.customer>`, `<x-layouts.tenant>`, dan `<x-layouts.admin>`. Pendekatan komponen menempatkan kerangka bersama pada berkas `resources/views/components/layouts/` dan mengisi konten melalui `{{ $slot }}`. Konsep pewarisan `@extends`/`@yield` tetap penting dipahami karena banyak dijumpai pada aplikasi Laravel yang lain.

Pewarisan layout mendukung prinsip *Don't Repeat Yourself* (DRY). Jika kerangka dasar memiliki *B* baris, setiap halaman memiliki *C* baris konten, dan terdapat *N* halaman, penghematan markup dapat diperkirakan dengan Persamaan 2.2.

$$L_{\text{tanpa}} = N\,(B + C), \qquad L_{\text{dengan}} = B + N\,C, \qquad \Delta L = (N-1)\,B \tag{2.2}$$

dengan *L*_tanpa total baris tanpa layout bersama, *L*_dengan total baris dengan pewarisan layout, *B* baris kerangka dasar, *C* baris konten per halaman, dan *N* jumlah halaman; penghematan $\Delta L = (N-1)B$.

Contoh 2.2. Penghematan markup dengan layout bersama. Misalkan $B = 40$ baris kerangka dasar, $C = 15$ baris konten per halaman, dan $N = 12$ halaman. Tanpa layout bersama, L_tanpa = 12(40 + 15) = 660 baris. Dengan layout bersama, L_dengan = 40 + 12(15) = 220 baris. Penghematan $\Delta L = (12 - 1)(40) = 440$ baris (Gambar 2.4).



Gambar 2.4. Perbandingan jumlah baris markup dengan dan tanpa pewarisan layout (data ilustratif).

Sumber: perhitungan penulis dengan pgfplots (data ilustratif).

Hasil tersebut menunjukkan manfaat pengurangan duplikasi pada contoh ilustratif. Namun, penghematan baris bukan satu-satunya alasan menggunakan layout. Layout perlu digunakan ketika elemen bersama memang stabil dan dipakai ulang. Hierarki layout yang terlalu dalam justru dapat menyulitkan penelusuran.

> **Tip.** Batasi kedalaman pewarisan layout. Untuk banyak aplikasi, satu layout utama dan satu variasi per peran sudah cukup mudah dipahami.

2.6.6 Formulir dengan Blade

Formulir menerima data dari pengguna. Pada Laravel, formulir POST, PUT, PATCH, atau DELETE perlu mempertimbangkan keamanan CSRF dan umpan balik validasi. Kode 2.7 memperlihatkan formulir untuk memperbarui ketersediaan menu, selaras dengan *endpoint* `tenant.menus.update`.

Formulir pembaruan menu (Blade)

```
<form method="POST" action="{{ route('tenant.menus.update', [$tenant, $menu]) }}">
    @csrf
    @method('PATCH')

    <div>
        <label for="is_available">Ketersediaan</label>
        <select id="is_available" name="is_available" required>
            <option value="1" @selected(old('is_available', $menu-
>is_available))>Tersedia</option>
            <option value="0" @selected(! old('is_available', $menu-
>is_available))>Habis</option>
        </select>

        @error('is_available')
            <small class="galat">{{ $message }}</small>
        @enderror
    </div>
```

```
    <button type="submit">Simpan</button>
</form>
```

Kode 2.19. Formulir pembaruan menu pada konteks tenant. Formulir memakai `@method` untuk melakukan *method spoofing*, `@csrf` untuk token antipemalsuan, `old()` untuk mengembalikan input sebelumnya, serta `@error` untuk menampilkan pesan validasi.

Alur formulir tersebut: pengguna memilih ketersediaan; browser mengirim *request* POST yang di-*spoof* menjadi PATCH melalui `@method('PATCH')` ke *route* `tenant.menus.update`; `@csrf` menyertakan token untuk membantu mencegah pemalsuan *request* lintas situs; jika validasi server gagal, aplikasi dapat mengarahkan pengguna kembali ke formulir; `old()` menampilkan kembali input sebelumnya; dan `@error` menampilkan pesan kesalahan untuk *field* terkait.

> **Peringatan.** Atribut HTML seperti `required` membantu pengalaman pengguna di browser, tetapi tidak menggantikan validasi di server. Validasi server tetap wajib karena *request* dapat dikirim tanpa melalui formulir yang Anda buat. Pada proyek, validasi server dilakukan melalui `$request->validate([...])` atau kelas *Form Request* (dibahas pada Bab 5).

2.7 Middleware, Response, Session, dan Akses Multi-Peran

Bagian ini menyatukan komponen yang memastikan *request* diproses dengan aman dan menghasilkan respons yang sesuai bagi setiap peran pengguna.

2.7.1 Middleware sebagai pipeline

Middleware bekerja sebagai *pipeline*. *Request* melewati *middleware* sebelum *controller* dipanggil. Setelah *controller* menghasilkan *response*, *response* dapat kembali melalui *middleware* yang sama dalam arah sebaliknya (lihat kembali Gambar 2.1). *Middleware* cocok untuk urusan yang berlaku pada banyak *request*, seperti autentikasi, otorisasi awal, *logging*, pembatasan laju, dan penetapan tenant.

2.7.2 Middleware peran

Kode 2.20 menampilkan *middleware* pemeriksaan peran pada proyek.

app/Http/Middleware/EnsureUserHasRole.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

final class EnsureUserHasRole
{
    public function handle(Request $request, Closure $next, string $role): Response
    {
        $user = $request->user();

        // Guest ditangani middleware auth; di sini fokus pada role/status.
        if ($user === null || ! $user->hasRole($role)) {
            abort(403);
        }

        return $next($request);
```

```
    }
}
```

Kode 2.20. *Middleware* `EnsureUserHasRole`. Middleware menerima nama peran dari definisi *route*, misalnya melalui `->middleware('role:admin')`. Jika pengguna belum login atau tidak memiliki peran yang sesuai, *middleware* menghentikan *request* dengan respons 403 (*Forbidden*). Jika pemeriksaan berhasil, *request* diteruskan melalui `$next($request)`.

Metode `hasRole()` pada contoh tersebut bukan metode bawaan Eloquent. Pada proyek Kantin, metode ini diimplementasikan pada model `User` sebagai kontrak peran interim: peran disimpan pada kolom `role` dan diperiksa bersama status aktif pengguna. Implementasi ringkasnya sebagai berikut.

```
app/Models/User.php  (kutipan)
public function hasRole(string $role): bool
{
    // Kontrak role interim (kolom `role`); diformalkan menjadi relasi pada Modul 4-5.
    return $this->role === $role && $this->isActive();
}
```

Catatan. Karena `hasRole()` adalah metode proyek, perilakunya dapat berkembang. Pada bab lanjut, peran diformalkan melalui relasi keanggotaan tenant dan *policy*. Yang penting untuk saat ini: pemeriksaan peran terjadi di server, bukan sekadar dengan menyembunyikan menu.

2.7.3 Autentikasi dan otorisasi

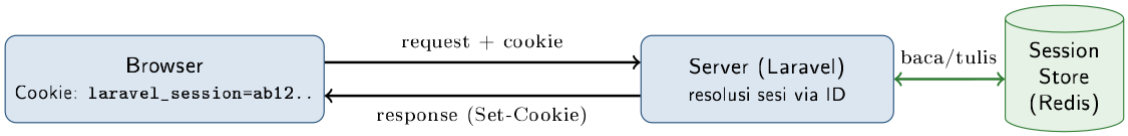
Autentikasi dan otorisasi memiliki fungsi yang berbeda. **Autentikasi** menjawab pertanyaan “siapa pengguna ini?”, sedangkan **otorisasi** menjawab “apa yang boleh dilakukan pengguna ini?”. Dalam aplikasi Kantin: admin dan operator tenant memerlukan autentikasi; pelanggan dapat menggunakan sesi anonim berbasis QR sesuai rancangan sistem; dan akses fitur diperiksa melalui *middleware*, *policy*, atau aturan server lainnya. Menyembunyikan menu navigasi bukanlah otorisasi. Pengguna tetap dapat mencoba mengakses URL secara langsung. Karena itu, *route* dan aksi yang dilindungi harus tetap memeriksa hak akses di server.

2.7.4 Response dan pola Post/Redirect/Get

Response adalah hasil yang dikirim server kepada klien. Bentuknya dapat berupa HTML, *redirect*, JSON, atau kode status tertentu. Setelah menyimpan data dari formulir, aplikasi lazim memakai pola *Post/Redirect/Get* (PRG): pengguna mengirim formulir dengan metode POST; server memproses dan menyimpan data; server mengirim *redirect* ke halaman daftar; pengguna melihat halaman hasil melalui *request* GET. Pola PRG membantu mengurangi risiko browser mengirim ulang formulir ketika halaman dimuat ulang. Contohnya, `return redirect()->route('tenant.menus.index')->with('sukses', 'Menu tersimpan.');`. Pesan `with('sukses', ...)` lazim disimpan sebagai *flash message*, yaitu pesan yang tersedia untuk *request* berikutnya.

2.7.5 Session

Session menyimpan data lintas-*request* per pengguna. Dalam rancangan aplikasi Kantin, *session* dapat digunakan untuk menyimpan konteks meja pelanggan atau pilihan tampilan. Gambar 2.5 mengilustrasikan siklus *session*: browser menyimpan ID *session* pada *cookie*, sedangkan datanya disimpan di server.



Gambar 2.5. Siklus session: cookie ID di klien dan penyimpanan data di server.

Sumber: diadaptasi penulis (ilustrasi konseptual).

Tabel 2.8 membandingkan beberapa mekanisme penyimpanan *state*.

Mekanisme	Lokasi utama	Cakupan	Masa hidup
Request	Memori selama request diproses	Satu request	Sangat singkat
Session	Penyimpanan session yang dikonfigurasi	Satu pengguna	Hingga kedaluwarsa atau dihapus
Cookie	Browser pengguna	Satu browser	Sesuai atribut cookie
Cache	Server atau layanan cache	Dapat dibagikan lintas pengguna	Sesuai TTL atau penghapusan

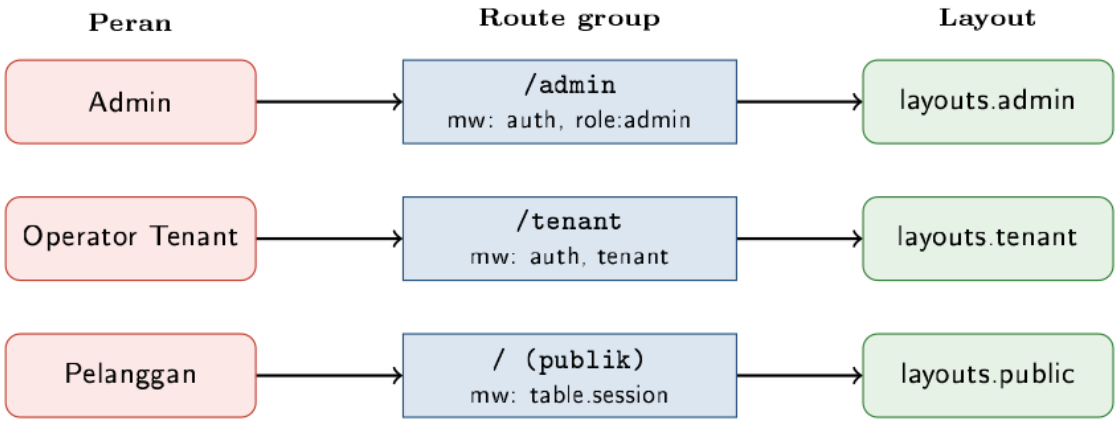
Tabel 2.8. Perbandingan mekanisme penyimpanan state.

Pada proyek Kantin, *driver* session disetel ke Redis melalui `SESSION_DRIVER=redis` pada berkas `.env`. Hal tersebut merupakan keputusan konfigurasi aplikasi, dan Laravel tetap mendukung *driver* session lain.

Peringatan. Jangan menyimpan data sensitif secara sembarangan di *session* atau *cookie*. Saat data *session* dipakai untuk menentukan akses atau ruang lingkup tenant, aplikasi tetap harus memverifikasi identitas, hak akses, dan kepemilikan data di server.

2.7.6 Pemetaan peran, route, dan layout

Agar pengalaman pengguna konsisten, setiap peran dapat dipetakan ke *route group* dan *layout* yang sesuai. Pemetaan ini ditunjukkan pada Gambar 2.6 dan dirangkum pada Tabel 2.9.



Gambar 2.6. Pemetaan peran ke route group dan layout multi-peran.

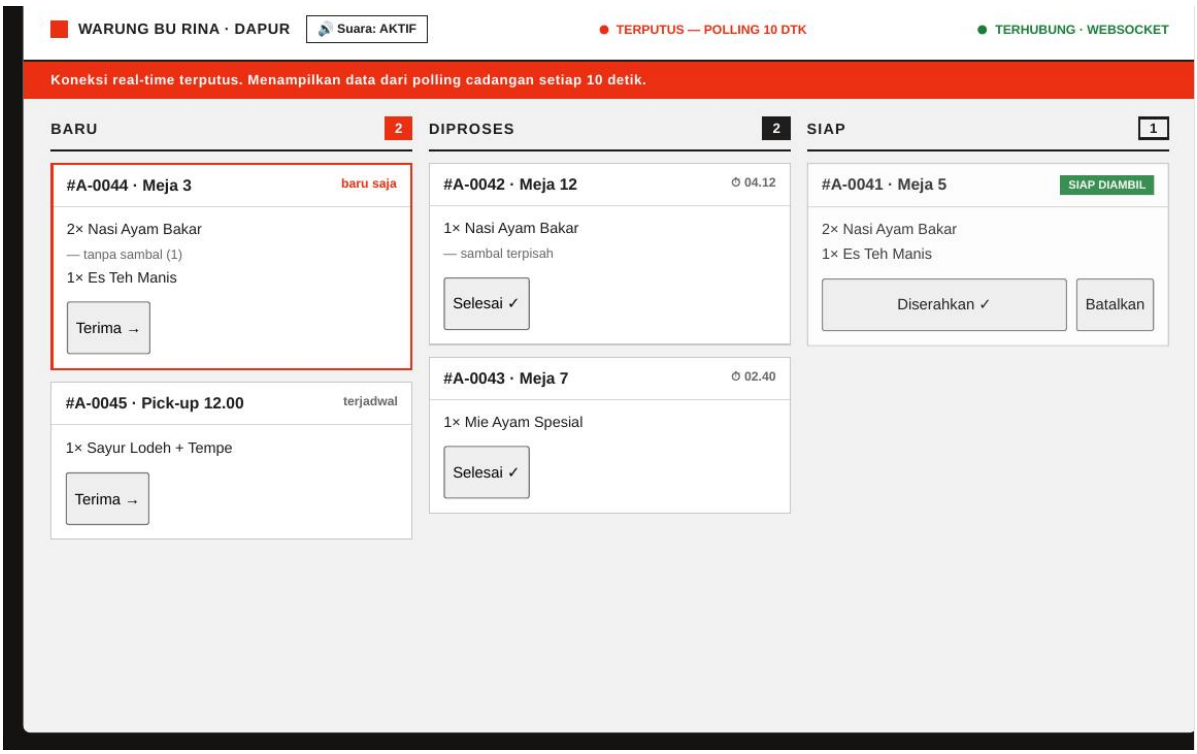
Sumber: diadaptasi penulis (ilustrasi konseptual).

Peran	Group route (prefix)	Middleware utama	Layout
Admin	admin	auth, verified, role:admin	x-layouts.admin

Operator tenant	tenant/{tenant:slug}	auth, verified, tenant	x-layouts.tenant
Pelanggan	kantin/{canteen}	web	x-layouts.customer

Tabel 2.9. Pemetaan peran, route, layout, dan navigasi.

Pemetaan tersebut membentuk rantai tanggung jawab: peran menentukan *route group*; *middleware* memeriksa akses; *controller* menjalankan tindakan aplikasi; dan *layout* serta navigasi menyajikan antarmuka yang sesuai. Gambar 2.7 memperlihatkan salah satu antarmuka operator tenant, yaitu *Kitchen Display System*.



Gambar 2.7. Antarmuka operator tenant (Kitchen Display System) aplikasi Kantin.

Sumber: berkas Mockup UI Kantin Multi-Tenant v2 (ilustratif).

2.8 Studi Kasus Terintegrasi: Akses Menu dan Pengelolaan Tenant

Bagian ini menghubungkan konsep-konsep dalam bab melalui dua skenario.

2.8.1 Skenario pelanggan membuka detail menu

Seorang pelanggan telah memindai QR meja dan memperoleh konteks meja yang valid. Pelanggan membuka halaman detail menu. Alurnya dapat dianalisis sebagai berikut.

1. Browser mengirim *request* GET ke URL detail menu pada konteks pelanggan.
2. *Route* pelanggan (grup `customer.`) menerima *request* tersebut.
3. *Middleware* konteks pelanggan memeriksa validitas sesi atau token meja.
4. Laravel melakukan *route model binding* untuk menghubungkan parameter menjadi objek `Menu`.
5. *Controller* menerima objek menu.
6. *Controller* memanggil layanan katalog (`PublicCatalogQuery`) untuk menyiapkan data bagi pelanggan.
7. *Controller* mengembalikan *view* dengan layout `x-layouts.customer`.

8. Blade menampilkan nama, harga, dan status stok menu.
9. Server mengirim respons HTML ke browser.

Analisis utama dari skenario ini adalah bahwa setiap bagian memiliki tugas berbeda. *Middleware* memeriksa konteks akses. *Controller* mengatur alur. *Service* menyiapkan data. *View* menampilkan data.

2.8.2 Skenario operator menambah/memperbarui menu

Operator tenant telah login dan membuka halaman pengelolaan menu. Operator lalu mengirim formulir pembaruan menu. Alurnya adalah sebagai berikut.

1. Operator mengakses *route* tenant melalui awalan `tenant/{tenant:slug}`.
2. *Middleware* `auth` dan `verified` memeriksa identitas operator.
3. *Middleware* `tenant` menetapkan dan memverifikasi konteks tenant, lalu `scopeBindings()` mengunci `{menu}` di bawah tenant.
4. Operator mengisi formulir yang memuat token CSRF dan *spoofing* metode PATCH.
5. Formulir mengirim *request* ke *route* `tenant.menus.update`.
6. *Controller* memeriksa *policy*, lalu memvalidasi input di server.
7. *Service* atau model menyimpan perubahan dalam ruang lingkup tenant yang tepat.
8. *Controller* mengembalikan respons (JSON atau *redirect* dengan pesan *flash*).
9. Halaman pengelolaan menggunakan layout `x-layouts.tenant` dan menampilkan menu milik tenant yang sedang aktif.

Pada skenario ini, dua risiko harus diperhatikan. Pertama, menyembunyikan menu admin dari operator tidak cukup; akses *route* admin harus tetap dibatasi di server melalui `role:admin`. Kedua, menerima `tenant_id` dari formulir tanpa verifikasi dapat menimbulkan risiko akses silang antar-tenant; tenant harus ditentukan atau diverifikasi melalui konteks server yang tepercaya (`tenant` *middleware* dan `scopeBindings()`), bukan dari input pengguna.

Rangkuman Bab

- Aplikasi multi-peran memerlukan pembagian tanggung jawab agar fitur lebih mudah dipahami, diuji, dan dikembangkan.
- Alur dasar Laravel terdiri atas *request*, *route*, *middleware*, *controller*, *service* atau *model*, kemudian *view* atau *response*.
- *Modular monolith* menjalankan aplikasi sebagai satu unit *deployment*, tetapi membagi kode menjadi modul dengan batas domain yang jelas.
- *Bounded context* membantu menjaga agar istilah, aturan, dan data digunakan secara konsisten dalam batas tanggung jawabnya.
- *Route* menentukan tujuan *request*; *middleware* menangani pemeriksaan lintas-*request*; *controller* mengoordinasikan proses; *service* menyimpan logika bisnis; dan Blade menyajikan data.
- *Route model binding* menyederhanakan pengambilan model, tetapi tidak menggantikan pembatasan tenant dan otorisasi.
- Blade membantu membangun *view* yang terstruktur melalui direktif, komponen, layout, serta dukungan formulir dan CSRF.

- *Session* membantu menyimpan *state* lintas-*request*, tetapi tidak boleh menggantikan verifikasi keamanan di server.
- Navigasi berbasis peran meningkatkan pengalaman pengguna, sedangkan otorisasi tetap harus ditegakkan oleh *middleware*, *policy*, dan aturan server.

Pada bab berikutnya, pembahasan dilanjutkan ke pengelolaan data, relasi model, serta strategi pembatasan data tenant agar desain modular pada bab ini dapat diterapkan secara aman dan konsisten.

Istilah Penting

Istilah	Makna ringkas
Bounded context	Batas tempat model domain, istilah, dan aturan berlaku secara konsisten
Modular monolith	Aplikasi satu deployment yang dibagi menjadi modul berbatas jelas
Service provider	Titik registrasi binding dan konfigurasi aplikasi atau modul
Dependency injection	Teknik menyediakan dependensi dari luar objek
Route model binding	Penyelesaian parameter route menjadi objek model
Middleware	Lapisan pemrosesan request sebelum atau sesudah controller
Blade component	Unit tampilan berparameter yang dapat dipakai ulang
Layout inheritance	Pewarisan kerangka halaman melalui Blade
Session	Data lintas-request per pengguna
Autentikasi	Proses memverifikasi identitas pengguna
Otorisasi	Proses memeriksa hak pengguna terhadap aksi atau sumber daya

Tabel 2.10. Istilah penting Bab 2.

Pertanyaan dan Latihan

A. Pemahaman

1. Jelaskan mengapa aplikasi multi-peran tidak sebaiknya menempatkan seluruh logika pada *controller*.
2. Uraikan perbedaan *route*, *middleware*, *controller*, *service*, dan *view* dalam satu alur *request*.
3. Jelaskan perbedaan autentikasi dan otorisasi dengan menggunakan contoh admin dan operator tenant.
4. Apa yang dimaksud dengan *bounded context*? Jelaskan mengapa kata “pesanan” dapat memiliki penekanan berbeda pada konteks Pemesanan, Dapur, dan Pembayaran.
5. Mengapa menyembunyikan menu navigasi tidak dapat dianggap sebagai mekanisme otorisasi?

B. Penerapan

6. Buat definisi *route* untuk menampilkan daftar menu operator tenant pada awalan `tenant/{tenant:slug}`. Terapkan *middleware* yang sesuai berdasarkan rancangan bab ini, lalu beri nama *route* yang konsisten.
7. Buat metode *controller* `show()` yang menerima model `Menu` melalui *route model binding* dan mengembalikan respons detail menu.

8. Buat komponen Blade sederhana bernama `status-pesanan` yang menerima properti `status`. Komponen harus menampilkan label “Menunggu”, “Diproses”, atau “Selesai”.
9. Rancang formulir Blade untuk memperbarui harga menu. Sertakan `@csrf`, `@method('PATCH')`, penggunaan `old()`, dan `@error`.
10. Buat route publik `/artikel/{post:slug}` yang mengikat model `Post` melalui kolom `slug`, lalu tambahkan batasan agar slug hanya menerima huruf kecil, angka, dan tanda hubung.
11. Susun sebuah *route group* untuk konteks admin dengan awalan URL `admin`, awalan nama `admin.`, dan middleware `['auth', 'verified', 'role:admin']`. Sertakan minimal tiga route di dalamnya dan tuliskan nama lengkap tiap route.
12. Jelaskan perbedaan hasil antara `Route::get('/pengguna/{user}/artikel/{post}', ...)` dengan dan tanpa `scopeBindings()`. Sebutkan satu risiko keamanan yang muncul apabila pembatasan tersebut diabaikan.
13. Jelaskan data apa yang layak disimpan pada *session* untuk pelanggan Kantin dan data apa yang sebaiknya tidak disimpan tanpa pertimbangan keamanan tambahan.

C. Analisis

14. Modul Pembayaran dirujuk oleh satu modul lain dan merujuk pada tiga modul lain. Hitung nilai *instability* menggunakan Persamaan 2.1, kemudian jelaskan interpretasinya.
15. Sebuah modul memiliki $C_a = 4$ dan $C_e = 0$. Hitung nilai I. Apa implikasi nilai tersebut bagi perubahan pada modul itu?
16. Dengan $B = 50$, $C = 20$, dan $N = 8$, hitung nilai L_{tanpa}, L_{dengan}, dan ΔL berdasarkan Persamaan 2.2.
17. Analisis risiko jika *route* `/menus/{menu}` hanya menggunakan *route model binding* tanpa pembatasan tenant. Jelaskan minimal dua tindakan yang dapat digunakan untuk mengurangi risiko tersebut.
18. Rancang peta modul untuk aplikasi layanan pesan-antar makanan kampus. Tentukan minimal lima *bounded context*, pemilik tanggung jawab tiap konteks, serta contoh hubungan antarkonteks.

Daftar Pustaka

Buku

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Stauffer, M. (2023). *Laravel: Up & Running* (3rd ed.). O'Reilly Media.

Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.

Artikel Jurnal dan Prosiding

Martin, R. C. (1994). OO design quality metrics: An analysis of dependencies. *Workshop on Pragmatic and Theoretical Directions in Object-Oriented Software Metrics (OOPSLA)*.

Fielding, R. T., & Taylor, R. N. (2002). Principled design of the modern Web architecture. *ACM Transactions on Internet Technology, 2*(2), 115–150. https://doi.org/10.1145/514183.514185

Anderson, L. W., & Krathwohl, D. R. (Eds.). (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman.

Dokumentasi dan Standar Daring

Laravel LLC. (2026). *Routing; Controllers; Blade Templates; Middleware; Session — Laravel 13.x Documentation*. Diakses 17 Agustus 2026 dari https://laravel.com/docs/13.x

Laravel LLC. (2026). *Routing — Laravel Documentation*. Diakses 30 Agustus 2026 dari https://laravel.com/framework/docs/routing

Livewire. (2026). *Laravel Livewire Documentation (v4.x)*. Diakses 17 Agustus 2026 dari https://livewire.laravel.com/docs

OWASP Foundation. (2021). *Cross-Site Request Forgery Prevention Cheat Sheet*. Diakses 17 Agustus 2026 dari https://cheatsheetseries.owasp.org/

Internet Engineering Task Force. (2022). *RFC 9110: HTTP Semantics*. Diakses 17 Agustus 2026 dari https://www.rfc-editor.org/rfc/rfc9110

Sumber Proyek (Dokumen Internal)

Politeknik Negeri Banyuwangi. *SRS Aplikasi Kantin Multi-Tenant (ISO/IEC/IEEE 29148), versi 2*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *Mockup UI Kantin Multi-Tenant, versi 2*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *Modul Praktikum Kantin Multi-Tenant Laravel 13 (Mentor Tutorial)*. Dokumen praktikum.

Repositori proyek Kantin Multi-Tenant (Laravel 13). Berkas rujukan: `bootstrap/app.php`, `routes/tenant.php`, `app/Http/Controllers/Tenant/TenantMenuController.php`, `app/Http/Middleware/EnsureUserHasRole.php`, `app/Models/User.php`, `resources/views/components/status-badge.blade.php`.